

Test Plan and Report

Product Name: Vision Studio Spatial Layout Engine

Team Name: Vision Studio

Date: Jun 2, 2026

Testing Overview

Vision Studio Spatial Layout Engine was tested to verify that the final release supports the main user workflow: uploading or creating a room layout, editing the room, using AI-assisted design tools, browsing and placing furniture, detecting fit and collision problems, saving/loading projects, viewing the room in 2D and 3D, and exporting layout data.

Testing was completed using a combination of automated unit/integration tests, backend smoke tests, production build verification, and manual system test scenarios. The purpose of this test plan and report is to document what was tested, what passed, what remains limited, and what should be improved in future work.

Test Environment

1. Frontend
 - 1.1. React + Vite
 - 1.2. TypeScript
 - 1.3. Tailwind CSS
 - 1.4. Konva.js for 2D editor interactions
 - 1.5. Three.js / React Three Fiber for 3D visualization
2. Backend/Data
 - 2.1. Node.js + Express
 - 2.2. Supabase for database/auth/storage
 - 2.3. Server API routes for room, project, furniture, chat, export, and recognition flows
3. AI/Python Service
 - 3.1. Python / FastAPI
 - 3.2. OpenCV and AI-assisted parsing tools
 - 3.3. Floor plan and room photo parsing support
4. Deployment/Runtime Assumptions
 - 4.1. Frontend can be deployed on Vercel
 - 4.2. Express server and Python service can be deployed on Render
 - 4.3. Supabase credentials and schema must be configured for full save/load behavior

- 4.4. Some tests that require real Supabase credentials may be skipped or limited in local test environments

Testing Goals

1. Verify that the application can be installed, built, and run.
2. Verify that the frontend renders the core user interface without critical runtime errors.
3. Verify that the backend API responds correctly to core smoke test scenarios.
4. Verify that key user flows work manually from the user's perspective.
5. Verify that furniture placement, movement, and layout validation behave as expected.
6. Verify that the system can detect major fit and collision issues.
7. Verify that save/load behavior works when Supabase is configured.
8. Verify that the 3D preview and export flow are functional at an MVP level.
9. Identify known bugs, missing functionality, and edge cases for the release summary.

System Test Scenarios

Scenario 1: Application Startup

Purpose: Confirm that all major services can run and support the final application.

Steps:

1. Start the Express backend server.
2. Start the Python/FastAPI AI service.
3. Start the React/Vite frontend.
4. Open the frontend in the browser.
5. Confirm that the homepage or app entry point loads.

Expected Result:

- The frontend loads successfully.
- The backend service is reachable.
- The Python service is reachable when AI parsing features are used.
- No critical startup error prevents the app from opening.

Result: Passed in local/development testing when environment variables and dependencies are configured.

Scenario 2: Upload Floor Plan or Room Image

Purpose: Confirm that a user can begin the design process with their real space.

Steps:

1. Navigate to the upload page.
2. Upload a supported floor plan or room image.
3. Confirm that the file preview appears.
4. Continue through the upload or parsing flow.
5. Confirm that the user can proceed toward the editor.

Expected Result:

- The uploaded file is accepted and previewed.
- The app routes the user into the room layout workflow.
- If parsing succeeds, an editable layout is created or prepared.
- If parsing fails, the app should avoid crashing and give the user a fallback or error.

Result: Passed for supported upload/parsing scenarios. Known limitation: parsing accuracy depends on image quality, room shape, and AI service availability.

Scenario 3: 2D Room Editor Loads

Purpose: Confirm that the user can view and edit a room layout.

Steps:

1. Open an existing or newly created room layout.
2. Confirm that the editor page loads.
3. Confirm that the room layout appears on the 2D canvas.
4. Use available editor controls to adjust room layout elements.

Expected Result:

- The editor loads without a critical UI failure.
- The room layout appears in the workspace.
- User interaction with the editor is possible.
- Layout changes are reflected visually.

Result: Passed. The editor is functional for MVP room editing. Some complex geometry and advanced editing interactions remain limited.

Scenario 4: Chatbot / AI Design Guidance

Purpose: Confirm that a user can describe room goals and receive design assistance.

Steps:

1. Open the chatbot or AI assistant interface.
2. Enter a room design prompt or style request.
3. Submit the message.
4. Review the assistant's response.

Expected Result:

- The chatbot input accepts user text.
- The request is sent to the backend/AI service where configured.
- The user receives design-related guidance or layout suggestions.
- If the AI service is unavailable, the app should avoid crashing.

Result: Passed for configured AI/backend scenarios. Known limitation: AI output may be helpful, but is not guaranteed to be physically valid without additional fit/collision validation.

Scenario 5: Furniture Catalog Browsing

Purpose: Confirm that users can browse available furniture items.

Steps:

1. Open the editor.
2. Open the furniture catalog panel.
3. Browse furniture categories such as sofa, bed, table, chair, desk, and storage.
4. Select a furniture item.

Expected Result:

- Furniture categories are visible.
- Furniture cards include item information such as name, category, dimensions, and preview, where available.
- Users can select furniture for placement.

Result: Passed. The starter furniture catalog is functional. Known limitation: it is not a full live provider marketplace.

Scenario 6: Furniture Placement and Editing

Purpose: Confirm that users can place, move, rotate, and resize furniture.

Steps:

1. Select a furniture item from the catalog.
2. Place the item into the room layout.
3. Move the item to another location.
4. Rotate or resize the item where supported.
5. Confirm that the furniture state updates in the editor.

Expected Result:

- Furniture appears in the room after selection/placement.
- Furniture position updates when moved.
- Rotation and resizing controls update the item where supported.
- Placed furniture data is preserved in the layout state.

Result: Passed. Furniture placement and editing work for the MVP workflow.

Scenario 7: Fit and Collision Checks

Purpose: Confirm that the system catches major layout mistakes.

Steps:

1. Place furniture near or outside room boundaries.
2. Place two furniture items so they overlap.
3. Resize furniture so it becomes too large for the room.
4. Observe visual warnings or error messages.
5. Move or resize the furniture to resolve the issue.

Expected Result:

- The system detects obvious boundary and overlap problems.
- The user receives a warning or error message.
- Warnings are reduced or removed when the user corrects the issue.

Result: Passed for core collision and fit-checking cases. Known limitation: unusual room shapes, dense layouts, and custom dimensions may still create edge cases.

Scenario 8: Recommendations and Auto-Layout

Purpose: Confirm that the system can help users explore design ideas.

Steps:

1. Open a room layout in the editor.
2. View the recommendation panel or the suggested furniture section.

3. Trigger the auto-layout feature if available.
4. Review the generated layout.
5. Accept, reject, or manually adjust the result.

Expected Result:

- The system provides useful recommendations based on room or furniture context.
- Auto-layout generates a proposed layout.
- The generated layout attempts to avoid obvious room and furniture conflicts.
- The user can manually adjust the result.

Result: Partially passed. Recommendations and auto-layout are functional at a prototype level, but generated layouts may still require manual correction and further constraint refinement.

Scenario 9: Save and Load Project

Purpose: Confirm that users can continue their work later.

Steps:

1. Create or open a room project.
2. Make layout or furniture changes.
3. Save the project.
4. Reload or return to the project.
5. Confirm that the saved room and furniture data are restored.

Expected Result:

- Project data is stored when the save is triggered.
- Saved projects can be loaded later.
- Important layout state, including furniture placement data, is preserved.
- If Supabase credentials are missing or invalid, the system should fail gracefully.

Result: Passed when Supabase is configured correctly. Automated save/load tests may skip or be limited without real Supabase credentials.

Scenario 10: 3D Room Preview

Purpose: Confirm that users can preview their design in 3D.

Steps:

1. Create or load a room layout.
2. Place furniture items in the 2D editor.

3. Open the 3D preview.
4. Navigate the 3D scene using available camera controls.
5. Confirm that the room and furniture appear.

Expected Result:

- The room renders in 3D.
- Furniture appears in the scene as 3D models or placeholder geometry where supported.
- Camera controls allow the user to view the room from different angles.
- The 3D scene provides a useful preview of the layout.

Result: Passed at MVP level. Known limitation: 3D rendering is not photorealistic, and some furniture models may be simplified.

Scenario 11: Export Layout

Purpose: Confirm that users can export layout data for use outside the app.

Steps:

1. Create or open a room layout.
2. Place and adjust furniture.
3. Use the export feature.
4. Download or generate the supported export file.
5. Confirm that the export includes room and furniture state.

Expected Result:

- The app exports the layout data.
- The exported file includes the core room and furniture state.
- Export feedback is clear to the user.
- Unsupported formats or failed exports do not crash the app.

Result: Passed at MVP level for supported export behavior. Known limitation: external tool compatibility and production-level GLB/GLTF export reliability may need additional validation.

Unit & Test Integration

Frontend Tests

Test Location: client/src/**/__tests__/

Test Framework: Vitest

Command:

```
cd client
npm test
```

Verified Result:

- 173 tests passed
- 27 test files passed

Coverage Area:

- UI components
- Editor behavior
- Furniture placement logic
- Layout state behavior
- Client-side integration behavior
- Rendering and workflow-related utilities were tested

Status: Passed.

Server Tests

Test Location: server/__tests__/

Test Framework: Vitest

Commands:

```
cd server
npm test
cd server
npm run test:e2e
```

Verified Result:

- Server e2e smoke tests: 12/12 passed
- Additional server tests are present for CORS origins, placement persistence, and save/load behavior

Coverage Area:

- API smoke checks
- CORS configuration
- Placement persistence behavior
- Save/load logic where configured

Status: Passed for verified server smoke tests. Save/load-related tests may skip or require real Supabase credentials depending on the environment setup.

Python Service Tests

Test Location: python/

Command: N/A

Result:

- No formal Python unit tests were included in the final test suite.
- Python/FastAPI behavior was tested through manual system testing and integration with the application workflow.

Status: Not covered by automated tests in this release.

Build Verification

Client Production Build

Command:

cd client

npm run build

Expected Result:

- The frontend production build completes successfully.

Verified Result:

- Client production build succeeded.

Status: Passed.

Known Testing Limitations

1. AI parsing is difficult to fully test automatically.
 - 1.1. Floor plan parsing depends on image quality, room complexity, and AI service behavior, so some validation was completed manually.
2. Python service lacks formal unit tests.
 - 2.1. The Python/FastAPI service was tested through integration and manual workflows, but no dedicated Python test suite was included.
3. Save/load tests may depend on Supabase credentials.
 - 3.1. Some save/load behavior requires real Supabase configuration, so automated tests may skip or be limited in local environments.
4. 3D visualization was tested as an MVP preview.
 - 4.1. The 3D scene was tested for usability and basic correctness, but not for photorealistic accuracy or professional rendering quality.

5. Browser end-to-end tests are limited.
 - 5.1. Manual QA covered major user paths, but a full Playwright/Cypress-style browser test suite was not included in the final release.
6. External export compatibility needs more testing.
 - 6.1. Layout export works for supported MVP behavior, but future work should test more external tools and larger project files.

Last Modified: Jun 6, 2026